

CASE STUDY DOCUMENT

Blogging Platform with

Authentication and Admin Panel

InkWell

Write. Publish. Connect. Inspire.

Version 1.0 | 2026

1. Synopsis

InkWell is a full-stack Blogging Platform with Authentication and an Admin Panel that enables writers to publish rich content, build a readership, and manage their publishing workflow — all from a unified platform. The system supports three primary roles: Guests and Readers (who browse and engage with content), Authors (who create and manage their own posts), and Administrators (who oversee the entire platform).

Readers can browse a curated feed of published posts, filter by category or tag, search by keyword, read full articles, like posts, leave threaded comments, and subscribe to the newsletter. Authors get a full content management dashboard: a rich-text post editor with media upload, category and tag assignment, comment moderation for their own posts, and post analytics. Administrators manage users (with role assignment), moderate all content, manage the category and tag taxonomy, send newsletter campaigns, and access platform-wide analytics.

InkWell is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, post lifecycle management, comment and engagement management, category and tag taxonomy, media file management, newsletter and email subscriptions, and notifications — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Guests can browse published posts, view categories/tags, and read full articles without logging in.
- Users must register or log in (email/password or Google/GitHub OAuth) to comment, like posts, or subscribe.
- Role-based access control with three distinct roles: Reader, Author, and Admin.
- All users can update their profile, bio, avatar, and password.
- In-app and email notifications are dispatched for comment replies, new posts by followed authors, and newsletter campaigns.
- Admin panel provides full platform management: users, content, taxonomy, media, newsletter, and analytics.

2.2 Guest Requirements

- Browse the homepage feed of published posts in reverse-chronological order.
- View a full post by its SEO-friendly slug URL.
- Filter posts by category or tag.
- Search posts by keyword across titles and content.
- View author profiles and their published post listings.

- View the newsletter subscription widget and subscribe with an email address.
- Access the registration and login pages.

2.3 Reader Requirements

- Register and log in via email/password or Google/GitHub OAuth.
- Like a post; unlike a previously liked post.
- Add a top-level comment on any published post.
- Reply to an existing comment (threaded two-level discussion).
- Like a comment.
- Edit or delete own comments within an allowed time window.
- Subscribe to the newsletter with email and preference settings.
- Confirm the newsletter subscription via a double opt-in email link.
- Unsubscribe from the newsletter at any time via a one-click link.
- Receive in-app and email notifications for replies to own comments and @mentions.
- View, mark as read, and dismiss notifications from the notification centre.

2.4 Author Requirements

- Create posts using a rich-text editor with formatting, code blocks, and inline media.
- Save posts as drafts before publishing.
- Publish or unpublish posts at any time.
- Edit and update published or draft posts.
- Delete own posts (all associated comments are also removed).
- Upload images and other media files for use in posts; set alt text for accessibility.
- Assign a featured image to each post.
- Assign posts to one or more categories and tag them with relevant keywords.
- View and moderate comments on their own posts: approve pending comments, reject inappropriate ones, or delete comments.
- View basic post analytics: view count, like count, and comment count per post.
- Manage their personal media library.

2.5 Admin Requirements

- View and manage all user accounts: change roles (Reader/Author/Admin), suspend, reactivate, or permanently delete accounts.
- View and manage all posts: feature (pin to top of feed), moderate content, or delete any post.
- Create, edit, and delete categories including hierarchical (parent/child) structures.
- Create and delete tags; view trending tags by usage count.
- View and moderate all comments across the platform: approve, reject, or delete any comment.

- View all newsletter subscribers with status; send newsletter campaigns to all or filtered subscribers.
- View platform analytics: total posts, users, comments, most-viewed posts, most-active authors.
- Manage the site-wide media library: view all uploaded files, delete media.
- Send broadcast notifications to all users or by role.
- View audit logs of all significant admin actions.

2.6 Post Lifecycle Requirements

- Post statuses: DRAFT (only visible to author), PUBLISHED (public), UNPUBLISHED (hidden from public), ARCHIVED.
- Each post has a unique SEO-friendly slug auto-generated from the title (e.g., 'how-to-build-a-blog' from 'How to Build a Blog').
- Read time in minutes is computed server-side using the average reading speed (200 WPM) on save.
- View count is incremented atomically once per unique session to prevent inflation.
- Featured image is stored as a URL linking to the uploaded Media entity.

2.7 Comment Requirements

- Comments require authentication; guest comments are not allowed.
- Comments default to APPROVED status; admin can switch the platform to a moderation-required mode.
- Each comment stores parentCommentId (null for top-level) forming a two-level threaded structure.
- Deleting a parent comment also soft-deletes all child replies.
- Authors receive an in-app and email notification whenever a new comment is added to one of their posts.

2.8 Newsletter Requirements

- Double opt-in: subscribers receive a confirmation email with a unique token link before being activated.
- Each subscriber has a status: PENDING (awaiting confirmation), ACTIVE, or UNSUBSCRIBED.
- Newsletter campaigns can be targeted to all subscribers, or filtered by status or preference tags.
- A new-post notification email is sent automatically to ACTIVE subscribers when an author publishes a new post.
- Unsubscribe links in all emails use a unique per-subscriber token for one-click removal.

3. Use Case Diagram

The diagram below captures all actors and use cases within the InkWell platform — spanning authentication, public browsing, reader engagement, author content management, media handling, newsletter operations, notifications, and full administrative oversight.

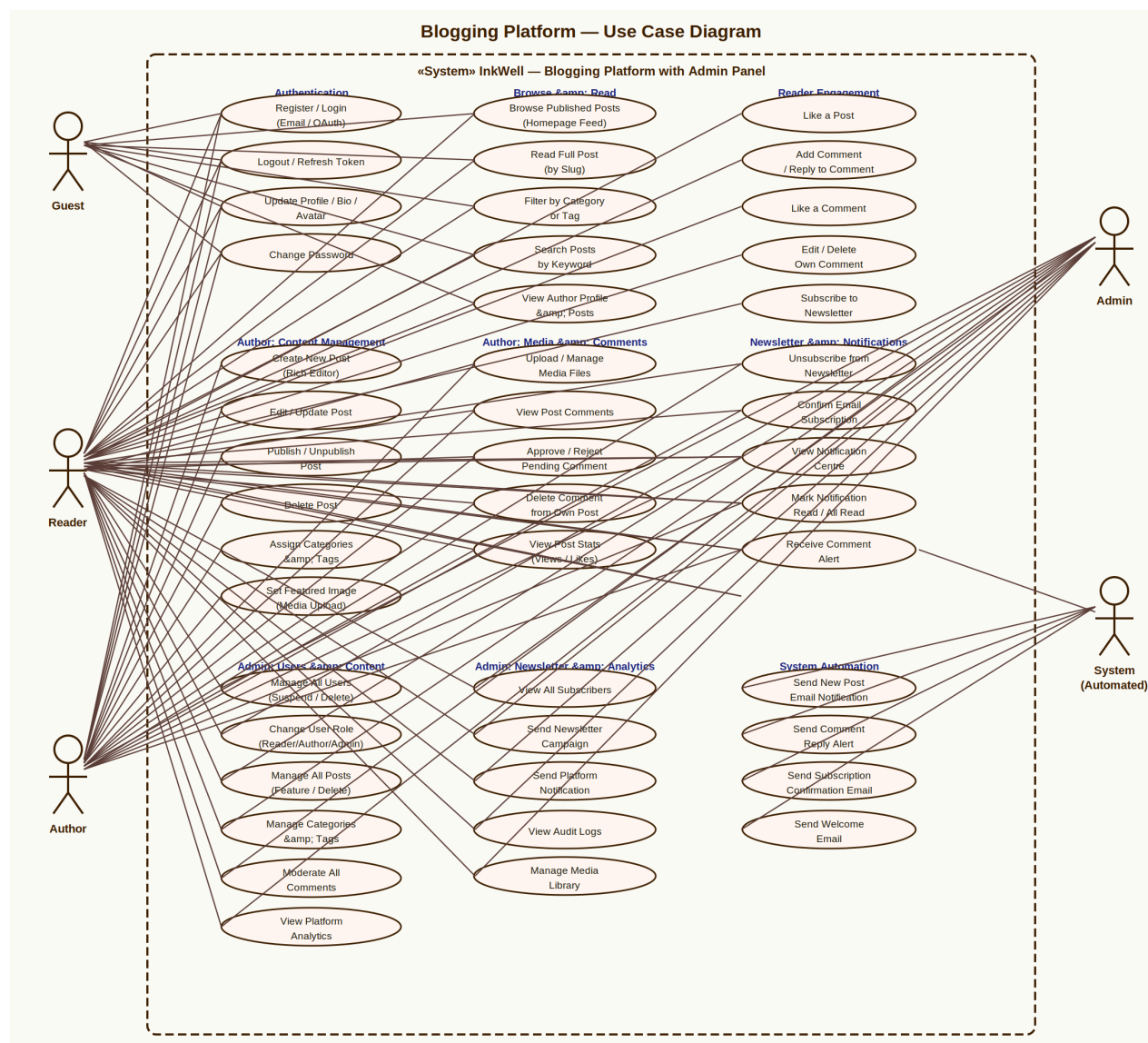


Figure 1: InkWell Blogging Platform — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can browse published posts, filter by category/tag, and search
Reader	Registered user who reads posts, likes content, comments, and subscribes to the newsletter
Author	Registered user who creates, edits, and publishes posts; moderates comments on own posts
Admin	Platform administrator who manages all users, content, taxonomy, media, and analytics
System (Automated)	Automated component dispatching new-post emails, comment alerts, and subscription confirmation emails

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	Guest, Reader, Author	Create account or authenticate via email or Google/GitHub OAuth
Update Profile / Bio / Avatar	Reader, Author	Change display name, bio text, avatar image, or email address
Change Password	Reader, Author	Update the account password via the profile settings page
Browse Published Posts	Guest, Reader	View the homepage feed of published posts in reverse-chronological order
Read Full Post	Guest, Reader	Open and read a full post article at its unique slug URL
Filter by Category or Tag	Guest, Reader	Narrow the post feed to a specific category or tag
Search Posts by Keyword	Guest, Reader	Full-text search across post titles and content
View Author Profile	Guest, Reader	See an author's bio, avatar, and list of published posts
Like a Post	Reader	Add a like reaction to a post; increment the post's likes counter
Add Comment / Reply	Reader, Author	Post a top-level comment or a threaded reply to an existing comment
Like a Comment	Reader, Author	Express approval on a specific comment
Edit / Delete Own Comment	Reader, Author	Modify or remove own comments within an allowed editing window
Subscribe to Newsletter	Guest, Reader	Submit an email address to join the newsletter; triggers double opt-in flow

Use Case	Actor(s)	Description
Confirm Email Subscription	Reader	Click the confirmation link in the welcome email to activate the subscription
Unsubscribe from Newsletter	Reader	One-click unsubscribe from all newsletter emails using the token link
Receive Comment Alert	Author, System	In-app and email notification when a new comment is added to own post
Mark Notification Read	Reader, Author	Dismiss individual or all unread notifications from the notification centre
Create New Post	Author	Open the rich-text editor and compose a new blog post
Edit / Update Post	Author	Modify the title, content, categories, tags, or featured image of any own post
Publish / Unpublish Post	Author	Toggle a post between publicly visible and hidden-from-public states
Delete Post	Author	Permanently remove a post and all its associated comments and media links
Assign Categories & Tags	Author	Link a post to one or more categories and keyword tags during editing
Upload Media File	Author	Upload images or documents to the media library for use in posts
Manage Own Media Library	Author	View, update alt text, or delete own uploaded files
View Post Comments	Author	See all comments on own posts with pending/approved/rejected status
Approve / Reject Comment	Author, Admin	Approve a pending comment or reject an inappropriate one on own posts
View Post Stats	Author	Check view count, like count, and comment count per published post
Manage All Users	Admin	View, change roles, suspend, reactivate, or delete any user account
Feature / Delete Post	Admin	Pin a post to the top of the feed or remove any post for policy violations
Manage Categories & Tags	Admin	Create hierarchical categories and tags; delete unused taxonomy entries
Moderate All Comments	Admin	Approve, reject, or permanently delete any comment across the platform
View All Subscribers	Admin	See the full newsletter subscriber list with status and preferences

Use Case	Actor(s)	Description
Send Newsletter Campaign	Admin	Compose and dispatch a newsletter email to all or filtered subscribers
View Platform Analytics	Admin	Access total post, user, comment, and engagement metrics
Manage Media Library	Admin	View all uploaded media platform-wide; delete files and clean up orphaned assets
Send Platform Notification	Admin	Broadcast an in-app or email alert to all users or by role
View Audit Logs	Admin	Inspect timestamped record of all significant user and admin actions
Send New Post Email	System	Auto-dispatch newsletter email to ACTIVE subscribers when a post is published
Send Comment Reply Alert	System	Dispatch in-app and email notification to the commenter on a new reply
Send Subscription Confirm	System	Email double opt-in confirmation link to a new newsletter subscriber
Send Welcome Email	System	Send a welcome email to a subscriber after they confirm their subscription

4. Class Diagrams — All Services

InkWell follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The nine class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service is the security and identity foundation of the platform. It manages registration, login, JWT token lifecycle, OAuth2 provider integration (Google/GitHub), profile management, and account deactivation. The User entity carries a role field (READER/AUTHOR/ADMIN) that gates access to the author dashboard and admin panel across all services. Admin can escalate any user to AUTHOR or ADMIN role from the admin panel.

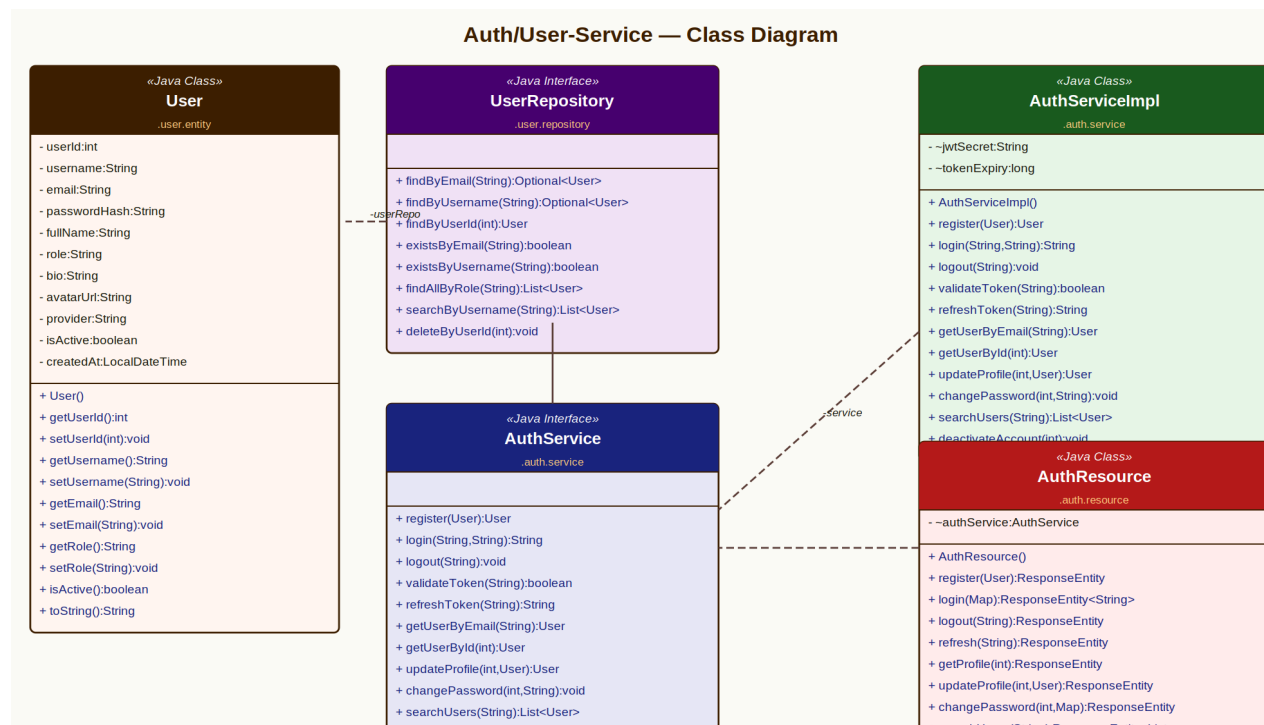


Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, username, email, passwordHash, fullName, role (READER/AUTHOR/ADMIN), bio, avatarUrl, provider (LOCAL/GOOGLE/GITHUB), isActive, createdAt
UserRepository	Java Interface	findByEmail(), findByUsername(), findByUserId(), existsByEmail(), existsByUsername(), findAllByRole(), searchByUsername(), deleteByUserId()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation/validation, OAuth2 handling, profile update, password change, user search, account deactivation

Component	Type	Responsibility
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserByEmail(), getUserById(), updateProfile(), changePassword(), searchUsers(), deactivateAccount()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/search, /auth/deactivate

4.2 Post-Service

The Post-Service manages the full lifecycle of blog posts. Each Post has a unique URL slug auto-generated from the title, a status (DRAFT/PUBLISHED/UNPUBLISHED/ARCHIVED), a computed read-time field, and denormalised like and view counters for efficient feed rendering. The publishPost operation sets the publishedAt timestamp and triggers a Newsletter-Service notification. View counts are incremented atomically using database-level increment operations to prevent concurrent update conflicts.

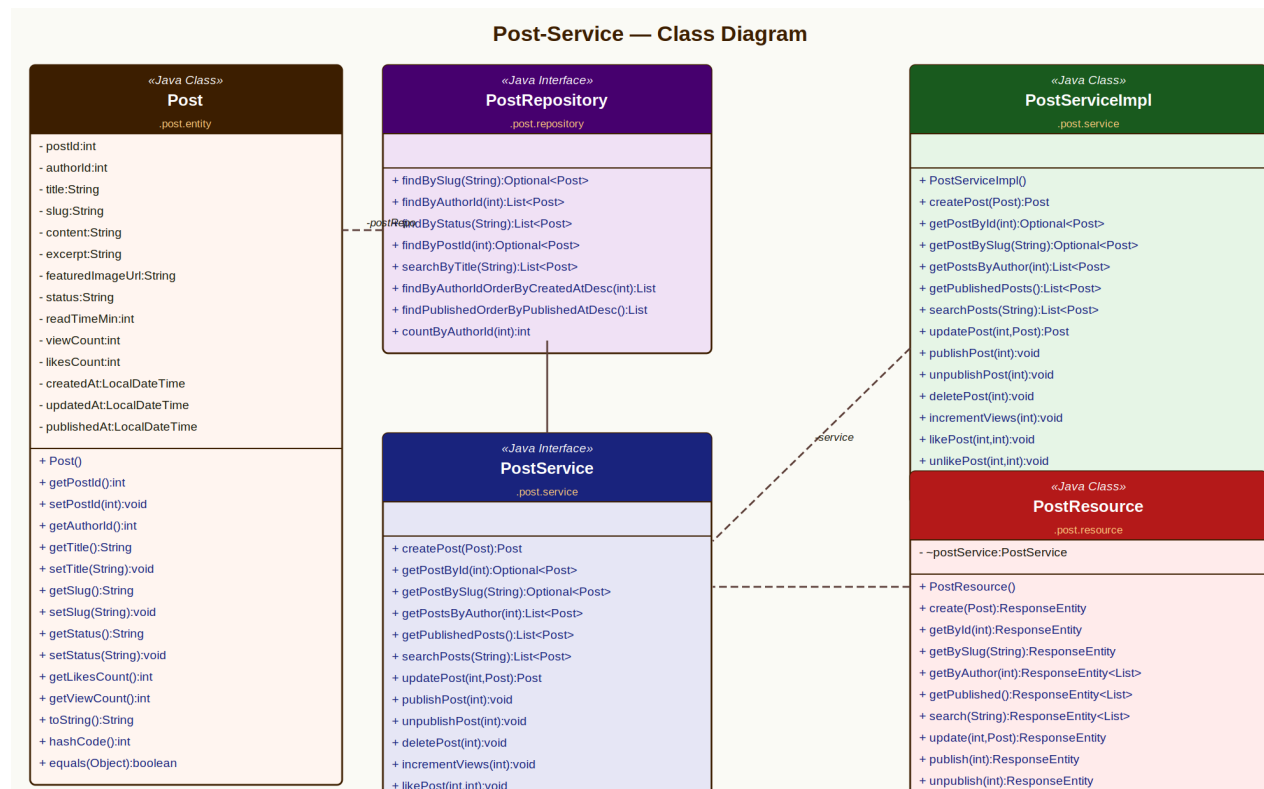


Figure 3: Post-Service — Class Diagram

Component	Type	Key Attributes / Methods
Post	Java Class (Entity)	postId, authorId, title, slug (unique), content (rich HTML), excerpt, featuredImageUrl, status (DRAFT/PUBLISHED/UNPUBLISHED/ARCHIVED), readTimeMin, viewCount, likesCount, createdAt, updatedAt, publishedAt
PostRepository	Java Interface	findBySlug(), findByAuthorId(), findByStatus(), findByPostId(), searchByTitle(), findByAuthorIdOrderByCreatedAtDesc(), findPublishedOrderByPublishedAtDesc(), countByAuthorId()

Component	Type	Key Attributes / Methods
PostServiceImpl	Java Class	createPost(), getPostById(), getPostBySlug(), getPostsByAuthor(), getPublishedPosts(), searchPosts(), updatePost(), publishPost(), unpublishPost(), deletePost(), incrementViews(), likePost(), unlikePost(), getPostCount()
PostService	Java Interface	Declares all post CRUD, status management, search, view increment, and like/unlike operations
PostResource	Java Class (REST)	Exposes /posts endpoints: POST (create), GET (by id/slug/author/published/search), PUT (update/publish/unpublish), POST (like/unlike), DELETE, GET count

4.3 Comment-Service

The Comment-Service manages threaded discussions on posts. Top-level comments have a null `parentCommentId`; replies carry the parent's ID for two-level threading. The status field (APPROVED/PENDING/REJECTED) supports comment moderation — authors can moderate their own post's comments; admins can moderate everything. A soft-delete approach marks comments as DELETED to preserve threading context in the UI even after removal.

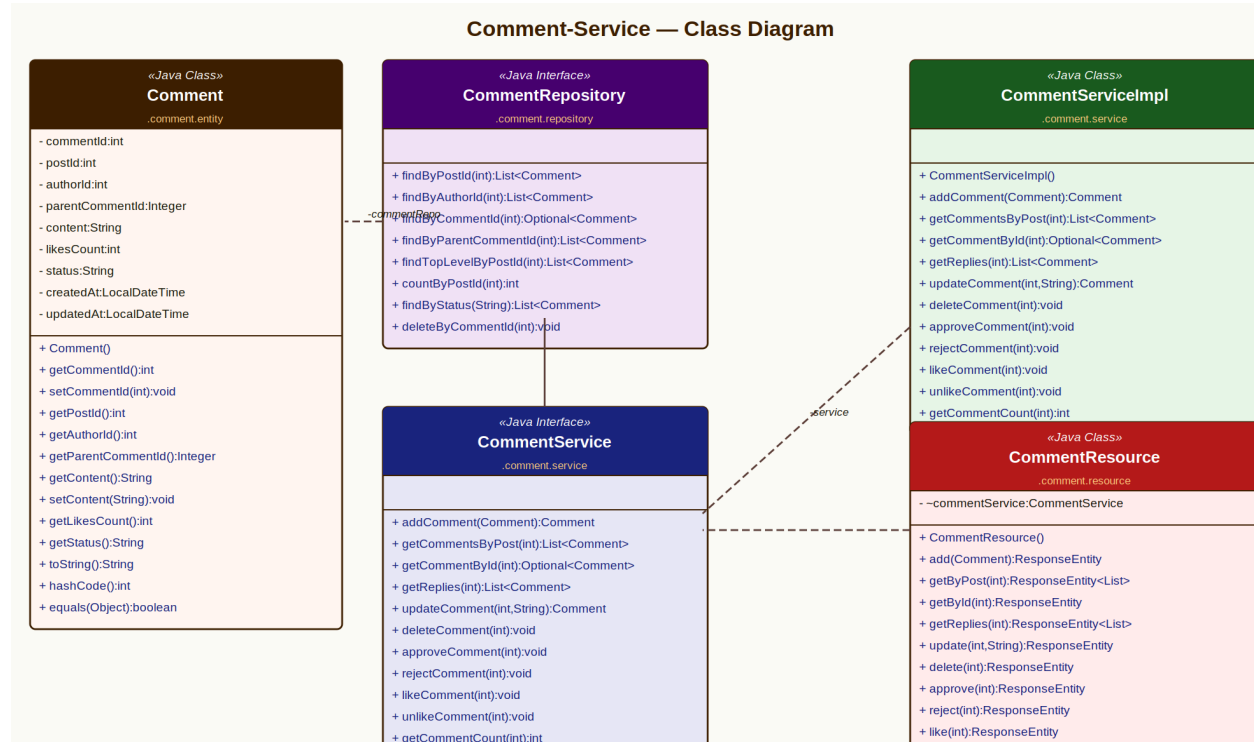


Figure 4: Comment-Service — Class Diagram

Component	Type	Key Attributes / Methods
Comment	Java Class (Entity)	<code>commentId</code> , <code>postId</code> , <code>authorId</code> , <code>parentCommentId</code> (nullable for top-level), <code>content</code> , <code>likesCount</code> , <code>status</code> (APPROVED/PENDING/REJECTED/DELETED), <code>createdAt</code> , <code>updatedAt</code>
CommentRepository	Java Interface	<code>findByPostId()</code> , <code>findByAuthorId()</code> , <code>findByCommentId()</code> , <code>findByParentCommentId()</code> , <code>findTopLevelByPostId()</code> , <code>countByPostId()</code> , <code>findByStatus()</code> , <code>deleteByCommentId()</code>
CommentServiceImpl	Java Class	<code>addComment()</code> , <code>getCommentsByPost()</code> , <code>getCommentById()</code> , <code>getReplies()</code> , <code>updateComment()</code> , <code>deleteComment()</code> , <code>approveComment()</code> , <code>rejectComment()</code> ,

Component	Type	Key Attributes / Methods
		likeComment(), unlikeComment(), getCommentCount()
CommentService	Java Interface	Declares all comment CRUD, reply threading, moderation (approve/reject), like/unlike, and count operations
CommentResource	Java Class (REST)	Exposes /comments endpoints: POST (add), GET (by post/id/replies), PUT (update/approve/reject/like/unlike), DELETE, GET count

4.4 Category & Tag-Service

The Category & Tag-Service manages the content taxonomy for the platform. Categories are hierarchical (parent/child) with a slug for SEO-friendly URLs. Tags are flat and accumulate a postCount for trending tag discovery. The service supports assigning and removing tags from posts, retrieving all tags for a post, and surfacing trending tags by usage frequency. Category and tag slugs are unique across the platform to prevent URL conflicts.

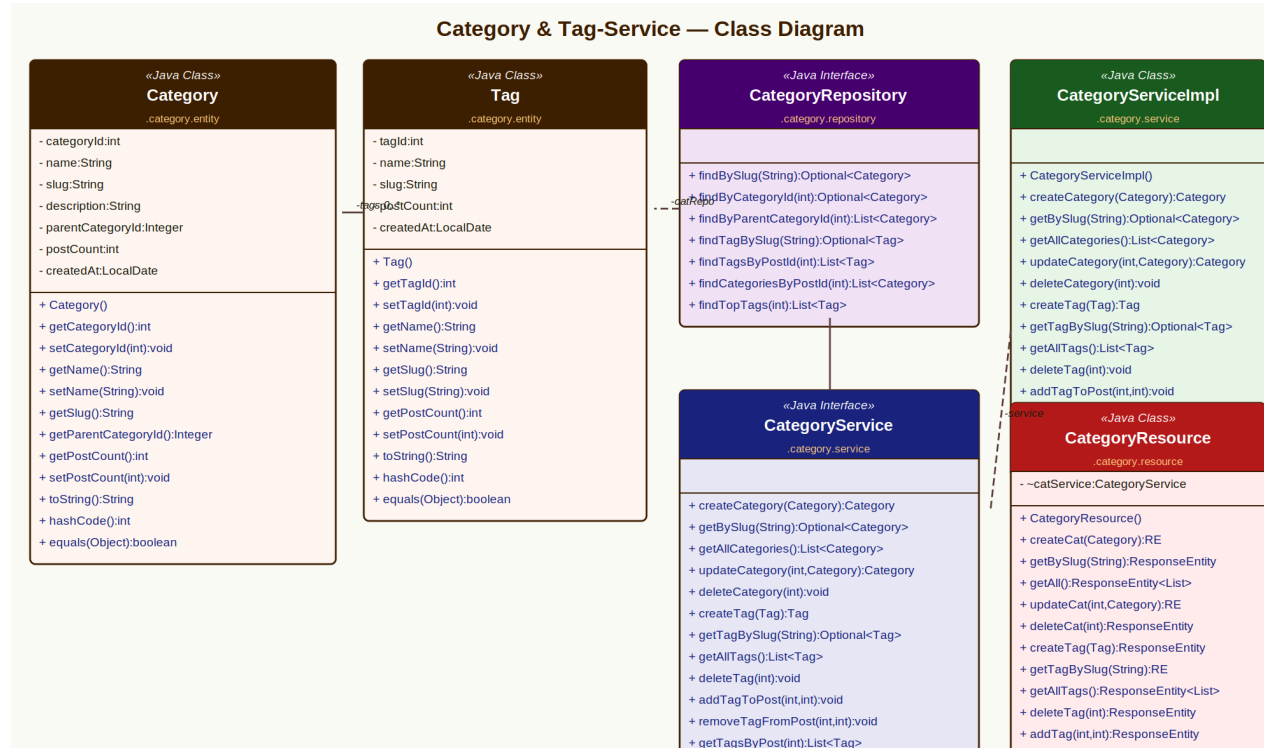


Figure 5: Category & Tag-Service — Class Diagram

Component	Type	Key Attributes / Methods
Category	Java Class (Entity)	categoryId, name, slug (unique), description, parentCategoryId (nullable for root), postCount, createdAt — supports hierarchical taxonomy
Tag	Java Class (Entity)	tagId, name, slug (unique), postCount, createdAt — flat taxonomy with usage-count tracking for trending
CategoryRepository	Java Interface	findBySlug(), findByCategoryId(), findByParentCategoryId(), findTagBySlug(), findTagsByPostId(), findCategoriesByPostId(), findTopTags()
CategoryServiceImpl	Java Class	createCategory(), getBySlug(), getAllCategories(), updateCategory(), deleteCategory(), createTag(),

Component	Type	Key Attributes / Methods
		getTagBySlug(), getAllTags(), deleteTag(), addTagToPost(), removeTagFromPost(), getTagsByPost(), getTrendingTags()
CategoryService	Java Interface	Declares all category CRUD, hierarchical navigation, tag CRUD, post-tag association, and trending tag operations
CategoryResource	Java Class (REST)	Exposes /categories and /tags endpoints: POST, GET (all/by slug/by post/trending), PUT, DELETE, POST/DELETE (add/remove tag from post)

4.5 Media-Service

The Media-Service handles file uploads for post featured images and inline media. Uploaded files are stored on AWS S3 and their metadata (filename, URL, MIME type, size, alt text) is persisted in the Media entity. The service supports linking a media file to a specific post and unlinking it when the post is deleted. A soft-delete flag preserves media records after deletion to maintain referential integrity in post content.

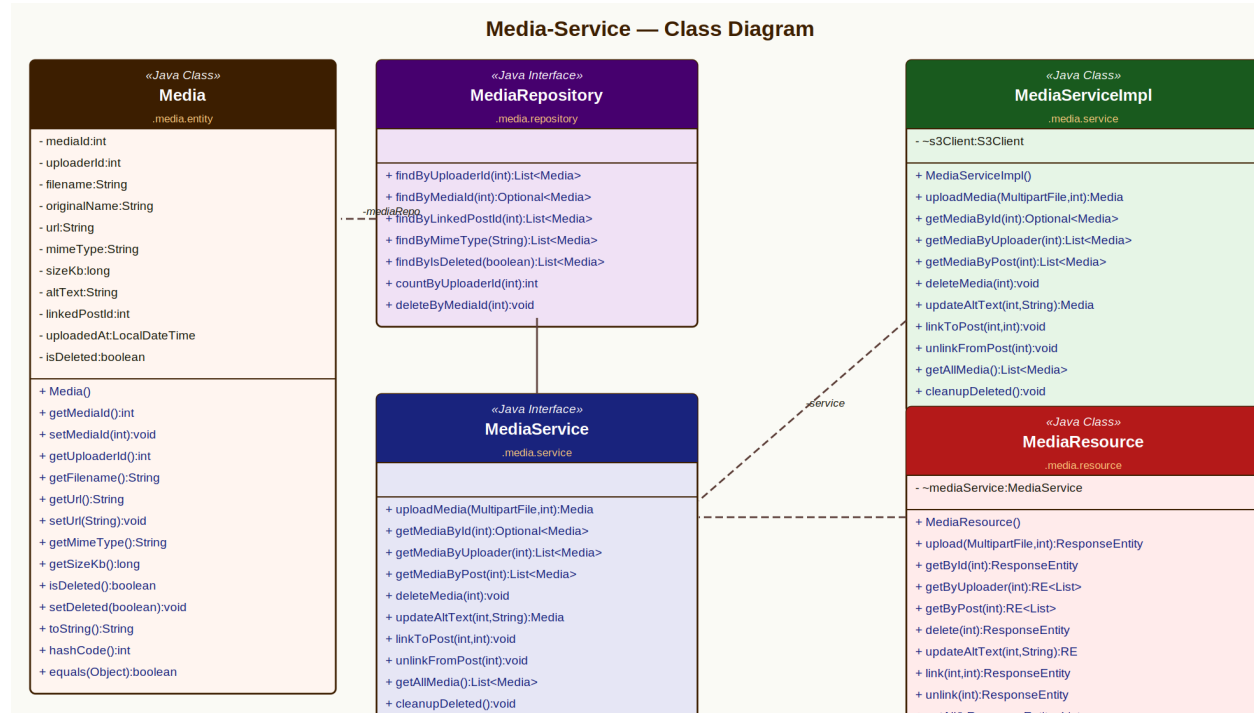


Figure 6: Media-Service — Class Diagram

Component	Type	Key Attributes / Methods
Media	Java Class (Entity)	mediaId, uploaderId, filename, originalName, url (S3 URL), mimeType, sizeKb, altText, linkedPostId, uploadedAt, isDeleted
MediaRepository	Java Interface	findByUploaderId(), findByMediaId(), findByLinkedPostId(), findByMimeType(), findByIsDeleted(), countByUploaderId(), deleteByMediaId()
MediaServiceImpl	Java Class	uploadMedia(), getMediaById(), getMediaByUploader(), getMediaByPost(), deleteMedia(), updateAltText(), linkToPost(), unlinkFromPost(), getAllMedia(), cleanupDeleted()
MediaService	Java Interface	Declares all media upload, retrieval, deletion, alt text update, post linking/unlinking, and cleanup operations

Component	Type	Key Attributes / Methods
MediaResource	Java Class (REST)	Exposes /media endpoints: POST (upload), GET (by id/uploader/post/all), PUT (altText), POST (link/unlink), DELETE

4.6 Newsletter / Subscription-Service

The Newsletter/Subscription-Service manages the email subscriber list and newsletter campaigns. The double opt-in flow creates a PENDING subscriber record and sends a confirmation email with a unique token. On token confirmation, the status changes to ACTIVE and a welcome email is sent. Newsletter campaigns can be targeted to all ACTIVE subscribers or filtered by preference tags. When an author publishes a new post, this service automatically dispatches a new-post notification to all ACTIVE subscribers.

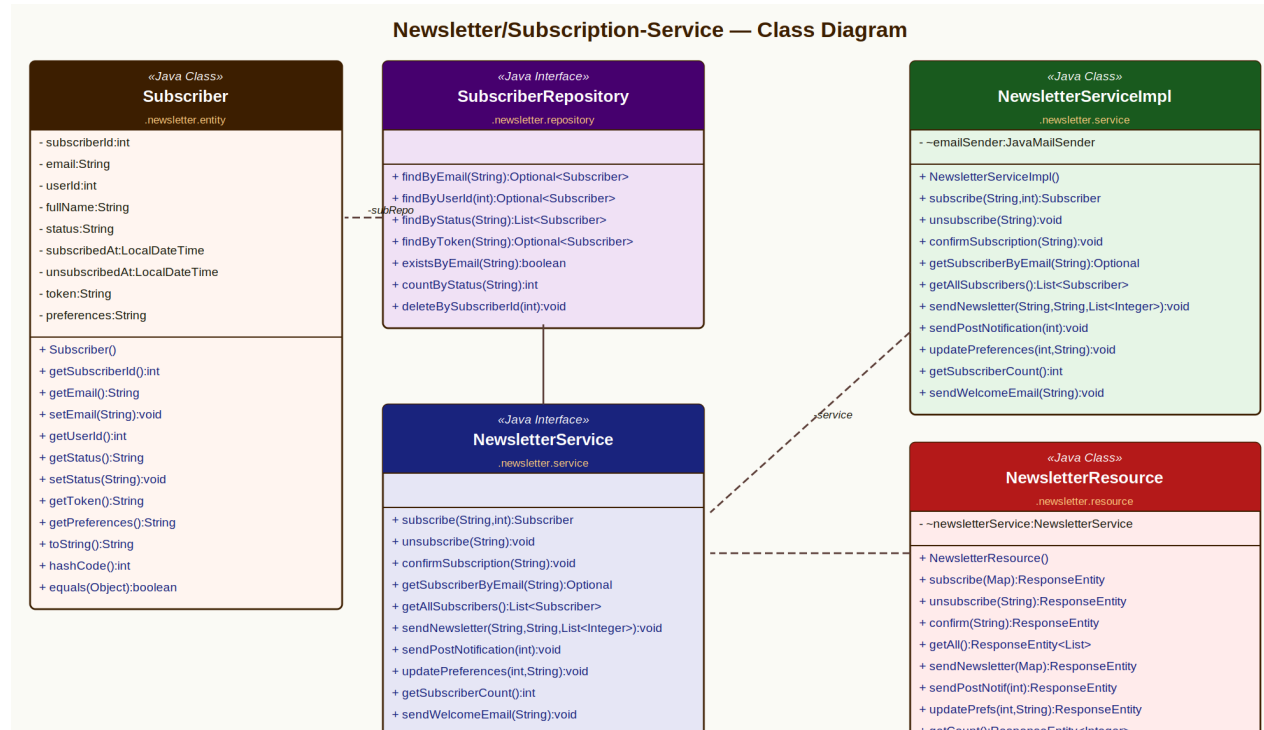


Figure 7: Newsletter/Subscription-Service — Class Diagram

Component	Type	Key Attributes / Methods
Subscriber	Java Class (Entity)	subscriberId, email, userId (nullable for non-registered subscribers), fullName, status (PENDING/ACTIVE/UNSUBSCRIBED), subscribedAt, unsubscribedAt, token (unique, for confirmation/unsubscribe), preferences
SubscriberRepository	Java Interface	findByEmail(), findByUserId(), findByStatus(), findByToken(), existsByEmail(), countByStatus(), deleteBySubscriberId()
NewsletterServiceImpl	Java Class	subscribe(), unsubscribe(), confirmSubscription(), getSubscriberByEmail(), getAllSubscribers(), sendNewsletter(), sendPostNotification(), updatePreferences(), getSubscriberCount(), sendWelcomeEmail()

Component	Type	Key Attributes / Methods
NewsletterService	Java Interface	Declares all subscription lifecycle (subscribe/confirm/unsubscribe), newsletter dispatch, post notification, preferences management, and count operations
NewsletterResource	Java Class (REST)	Exposes /newsletter endpoints: POST (subscribe), GET (confirm/unsubscribe via token), GET all (admin), POST (sendNewsletter/sendPostNotif), PUT (preferences), GET count

4.7 Notification-Service

The Notification-Service dispatches and manages in-app and email notifications for all key engagement events: new comment on author's post, reply to a comment, @mention in comment, new post published by a followed author, and admin broadcast messages. Notifications carry an actorId (who triggered the event) and relatedId/relatedType for deep-linking from the notification bell. The service tracks read/unread state and exposes an unread badge count.

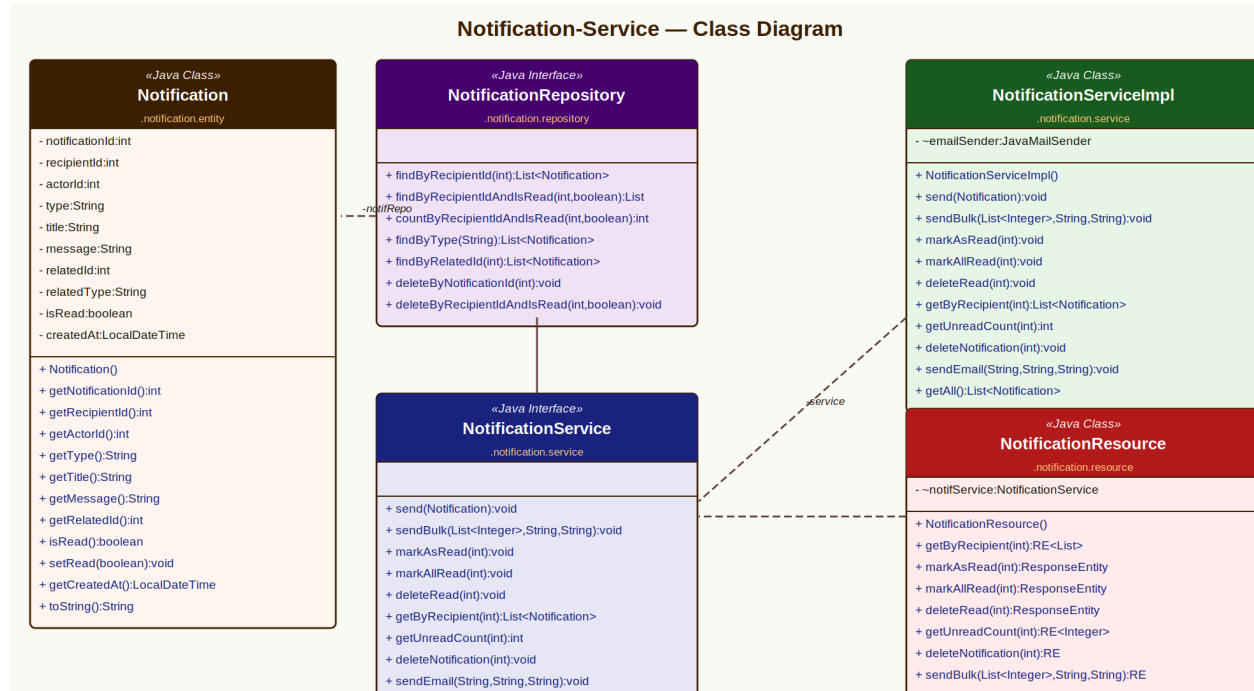


Figure 8: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, actorId, type (NEW_COMMENT/COMMENT_REPLY/MENTION/NEW_POST/LIKE), title, message, relatedId, relatedType, isRead, createdAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedId(), deleteByNotificationId(), deleteByRecipientIdAndIsRead()
NotificationServiceImpl	Java Class	send(), sendBulk(), markAsRead(), markAllRead(), deleteRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), getAll()
NotificationService	Java Interface	Declares notification dispatch (single/bulk/email), retrieval, read-state management, and deletion operations

Component	Type	Key Attributes / Methods
NotificationResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, deleteRead, getUnreadCount, delete, sendBulk, getAll

4.8 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices and renders Thymeleaf views for readers, authors, and administrators. It contains three controllers — BlogController (public-facing reader experience), AuthorController (author content management dashboard), and AdminController (admin panel) — each injecting relevant service dependencies via Spring @Autowired.

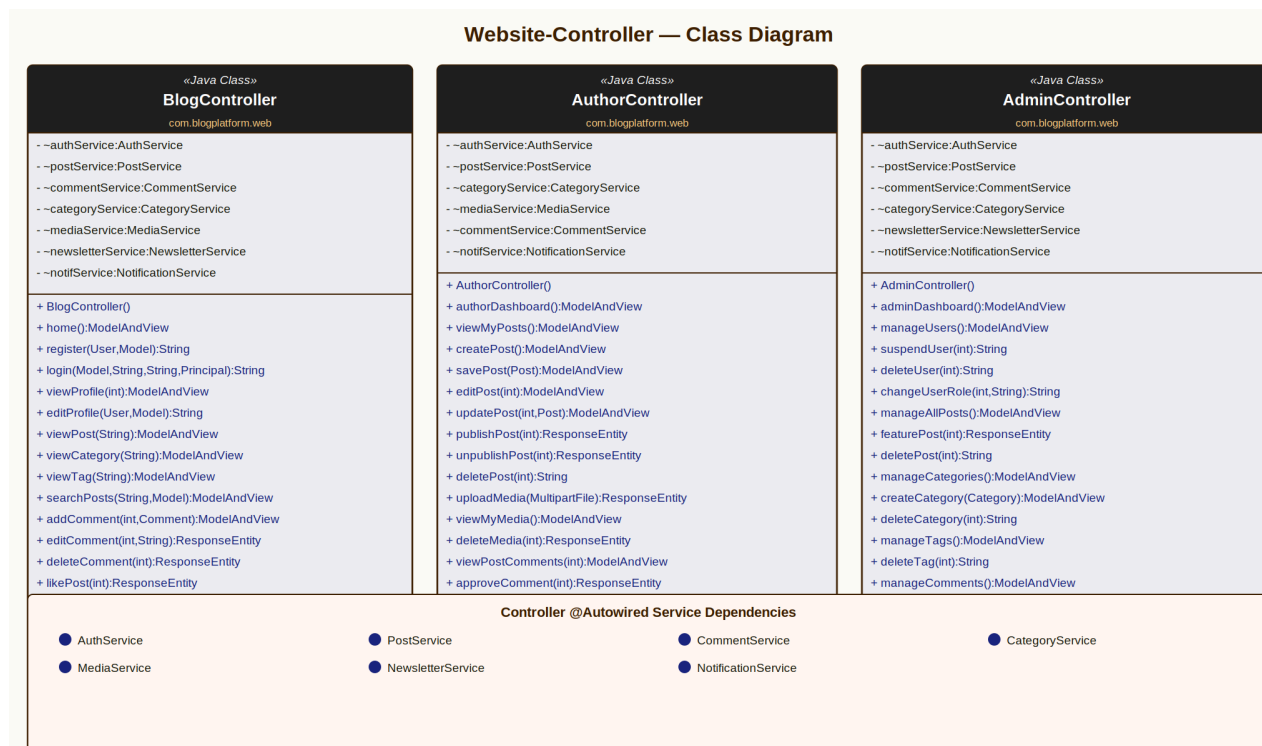


Figure 9: Website-Controller — Class Diagram

Controller	Type	Key Methods
BlogController	Java Class (MVC)	home(), register(), login(), viewProfile(), editProfile(), viewPost(), viewCategory(), viewTag(), searchPosts(), addComment(), editComment(), deleteComment(), likePost(), unlikePost(), likeComment(), subscribe(), unsubscribe(), viewNotifications(), markNotifRead(), logout()
AuthorController	Java Class (MVC)	authorDashboard(), viewMyPosts(), createPost(), savePost(), editPost(), updatePost(), publishPost(), unpublishPost(), deletePost(), uploadMedia(), viewMyMedia(), deleteMedia(), viewPostComments(), approveComment(), rejectComment(), viewMyNotifications()

Controller	Type	Key Methods
AdminController	Java Class (MVC)	adminDashboard(), manageUsers(), suspendUser(), deleteUser(), changeUserRole(), manageAllPosts(), featurePost(), deletePost(), manageCategories(), createCategory(), deleteCategory(), manageTags(), deleteTag(), manageComments(), approveComment(), deleteComment(), viewSubscribers(), sendNewsletter(), viewPlatformAnalytics(), sendPlatformNotification(), viewAuditLogs()

5. Microservices Architecture Overview

InkWell is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous newsletter dispatch and notification events are routed via RabbitMQ.

Microservice	Base Package	Primary Domain
auth-service	com.inkwell.auth	User accounts, JWT, OAuth2 (Google/GitHub), role management
post-service	com.inkwell.post	Post CRUD, status lifecycle, slugs, view/like counters
comment-service	com.inkwell.comment	Threaded comments, moderation (approve/reject), like/unlike
category-service	com.inkwell.category	Hierarchical categories, flat tags, post-tag associations, trending
media-service	com.inkwell.media	File upload to S3, media metadata, alt text, post linking
newsletter-service	com.inkwell.newsletter	Email subscriptions, double opt-in, newsletter campaigns, new-post alerts
notification-service	com.inkwell.notification	In-app and email notifications, bulk broadcast, read-state tracking
inkwell-web	com.inkwell.web	Spring MVC controllers, Thymeleaf view rendering

6. Non-Functional Requirements

Category	Requirement
Performance	Homepage feed must load within 1.5 seconds for 10,000 concurrent users; Redis caches the published post feed with 5-minute TTL
Scalability	Each microservice independently scalable via Docker/Kubernetes; static assets and media served via CloudFront CDN
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for social login; HTTPS enforced; XSS prevention via server-side HTML sanitisation of post content; CSRF protection on all state-changing endpoints
SEO	Each published post is accessible at /blog/{slug}; sitemap.xml auto-generated; Open Graph and Twitter Card meta tags rendered for social sharing previews
Availability	99.9% uptime SLA; health-check endpoints per service; graceful degradation if newsletter or notification service is temporarily unavailable
Comment Moderation	Platform-level flag controls whether new comments are auto-approved or enter PENDING status requiring author/admin approval
Newsletter Opt-In	Double opt-in is mandatory for GDPR compliance; confirmation tokens expire after 24 hours; unsubscribe links are unique per subscriber
Audit Trail	All admin actions (user suspension, post deletion, comment rejection, role change) are logged with actor, timestamp, and entity reference
Media Limits	Maximum file upload size: 10MB per file; supported formats: JPEG, PNG, GIF, WebP for images; PDF for documents; enforced at API Gateway
Usability	Responsive design for desktop, tablet, and mobile; WCAG 2.1 AA accessibility; dark mode support; reading progress indicator on long posts

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, Google OAuth App, GitHub OAuth App
Database	MySQL (primary relational store per service); Redis (post feed cache, session store, rate-limit counters)
Rich Text Editor	Quill.js or TipTap (ProseMirror-based) embedded in the author post editor for WYSIWYG editing
HTML Sanitisation	OWASP Java HTML Sanitizer to strip malicious HTML/scripts from user-submitted post content
File Storage	AWS S3 for media uploads; CloudFront CDN for fast image delivery; multipart upload via Spring MultipartFile
Email	Spring JavaMailSender via AWS SES for newsletter campaigns, comment alerts, subscription confirmation, and welcome emails
Messaging	RabbitMQ for asynchronous newsletter dispatch events, new-post notification fan-out, and comment alert queuing
Slug Generation	Server-side slugification library (e.g., Slugify) converting titles to URL-safe lowercase hyphenated slugs
SEO	Thymeleaf server-side rendering with Open Graph, Twitter Card, and JSON-LD structured data meta tags per post
Frontend	Thymeleaf templates with Bootstrap 5 (or React SPA with Next.js for SSR/SSG); syntax highlighting via Prism.js for code blocks
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling and rolling deployment
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints across all services
CI/CD	GitHub Actions pipeline: lint → test → Docker build → ECR push → Kubernetes rolling deploy

8. Glossary

Term	Definition
Post	A published or draft article written by an author, identified by a unique URL slug
Slug	A URL-safe, lowercase, hyphenated string derived from the post title used as the post's unique identifier in its URL (e.g., /blog/my-first-post)
Excerpt	A short auto-generated or manually written summary of a post displayed in feed listings
Read Time	An estimated reading duration in minutes computed server-side: word count divided by 200 WPM
Draft	A post in progress that is visible only to the author and not accessible to the public
Published	A post visible to all visitors at its slug URL and included in the public post feed
Featured Image	The primary visual image associated with a post, displayed in feed listings and social sharing previews
Category	A hierarchical taxonomy element grouping related posts; categories can have parent and child relationships
Tag	A flat keyword label attached to posts for discovery; tags accumulate a postCount for trending identification
Trending Tags	Tags ranked by their postCount, surfaced on the homepage to guide content discovery
Comment Thread	A two-level discussion structure where top-level comments (parentCommentId = null) can have one level of replies
Double Opt-In	A newsletter subscription confirmation method requiring the subscriber to click a unique link in a confirmation email before being activated
Subscriber Token	A UUID assigned to each newsletter subscriber for use in confirmation and unsubscribe links, enabling anonymous one-click actions
Newsletter Campaign	A composed email sent to all ACTIVE subscribers or a filtered subset, dispatched as an asynchronous job via RabbitMQ
Moderation Mode	A platform-level setting controlling whether new comments are auto-approved or enter a PENDING queue requiring explicit author or admin approval
Open Graph	A web metadata standard that defines how a post's title, image, and description appear when shared on social media platforms